



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Lab-MR 6

Navegación completa

Planificación global (Dijkstra/A) y navegación local reactiva por campos potenciales*

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: MATLAB

Índice

1. Objetivo	3
2. Implementación	3
3. Pruebas	4
3.1. Caso base: nodo 1 a la parte superior derecha	4
3.2. Zona conflictiva entre nodos 24 y 32	5
3.3. Reemplazo de Dijkstra por A*	5
4. Comentarios finales	6

1. Objetivo

Implementar un programa de MATLAB que combine la planificación global (Dijkstra de Lab-MR 4) con la navegación local reactiva por campos potenciales (Lab-MR 3) sobre el mapa `mapa2.pgm` y el grafo `mapa2.m`. El programa pide los nodos de inicio y destino, representa gráficamente el mapa con la trayectoria del robot e indica si se ha alcanzado el destino. Adicionalmente: realizar la trayectoria desde el nodo 1 hacia la zona superior derecha y atravesar la región entre los nodos 24 y 32 (donde la navegación básica falla por mínimo local), proponer una mejora, y reemplazar Dijkstra por A* con una heurística euclídea consistente.

2. Implementación

La solución es un único script (`navegacion_autonoma_p7.m`) que carga el mapa con `binaryOccupancyMap`, ejecuta `mapa2.m` para obtener nodos y costes, planifica la ruta global y recorre los subobjetivos con un bucle local de hasta 1200 iteraciones por tramo. Las decisiones no obvias son:

Matriz de costes euclídea para A*. A* solo es óptimo con heurística euclídea si los pesos del grafo coinciden con la métrica euclídea. Por eso, antes de invocar `aestrella`, se construye una matriz C^{eucl} que mantiene la topología (adyacencia) del grafo original pero sustituye cada peso $C(i, j)$ por $\|p_i - p_j\|_2$. Así la heurística $H(i, j) = \|p_i - p_j\|_2$ es consistente.

Campo tangencial para mitigar mínimos locales. Además de los campos atractivo y repulsivo de Lab-MR 3, se añade un campo tangencial F_{tan} orientado perpendicularmente al obstáculo más cercano, hacia el lado en el que se encuentra el subobjetivo (signo del producto vectorial entre la dirección al objetivo y la dirección al obstáculo). Su magnitud se modula por la cercanía $\max(0, 1 - d/D)$. Este campo desempata las situaciones de equilibrio entre atracción y repulsión.

Detección de estancamiento por progreso. Se mantiene una ventana deslizante con las últimas $W = 40$ distancias al subobjetivo. Si el progreso acumulado no supera un umbral Δ_{min} , se considera estancamiento. Este criterio es más robusto que medir velocidad nula porque el robot puede oscilar lateralmente a velocidad alta sin acercarse al objetivo.

Maniobra de escape. Al detectar estancamiento, se prueban $K = 16$ direcciones equiespaciadas alrededor de la dirección al subobjetivo, se descartan las que conducen a colisión y se selecciona la que maximiza una puntuación combinada

de despeje (mediana de las distancias láser tras desplazarse) y cercanía al subobjetivo. Tras un número máximo de estancamientos consecutivos, el tramo se aborta como mínimo local.

Detección de colisión por estencil de 9 puntos. Antes de cada avance se comprueba colisión no en un único punto, sino en el centro y en 8 puntos circundantes a distancia `collision_margin`. Esto modela un radio de seguridad y previene aproximaciones a esquinas que el campo continuo no detecta a tiempo.

3. Pruebas

3.1. Caso base: nodo 1 a la parte superior derecha

Con Dijkstra como planificador global y los campos potenciales básicos (sin campo tangencial ni maniobra de escape), el robot recorre la mayoría de los tramos sin incidencias y alcanza el destino en la zona superior derecha del mapa. Activando la mejora se obtiene un comportamiento equivalente, sin estancamientos, con una traza prácticamente idéntica.

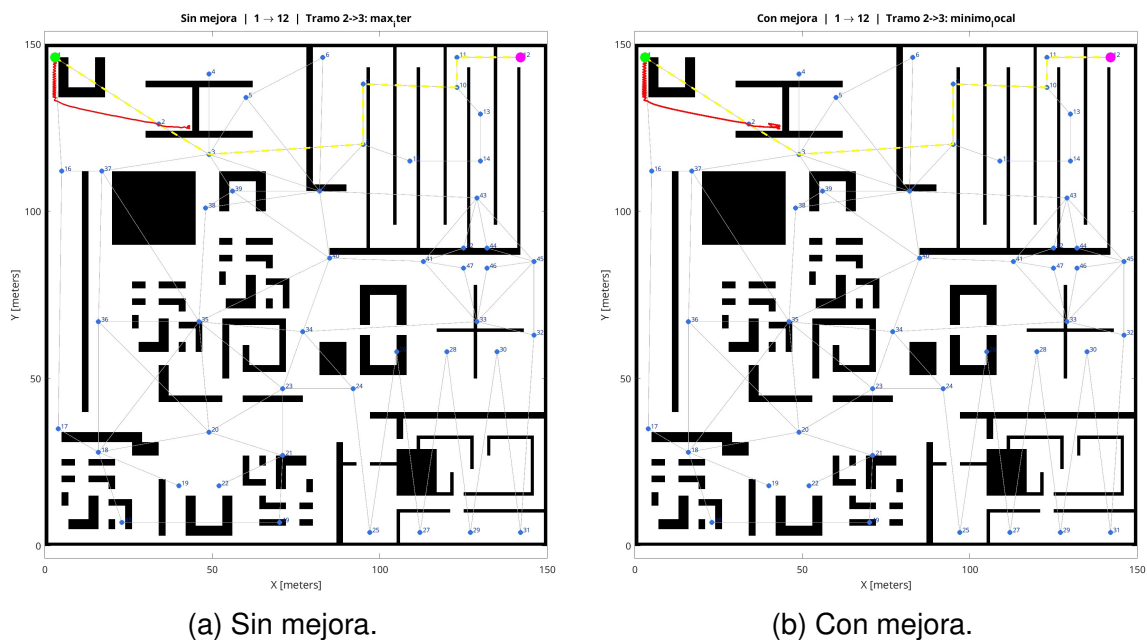
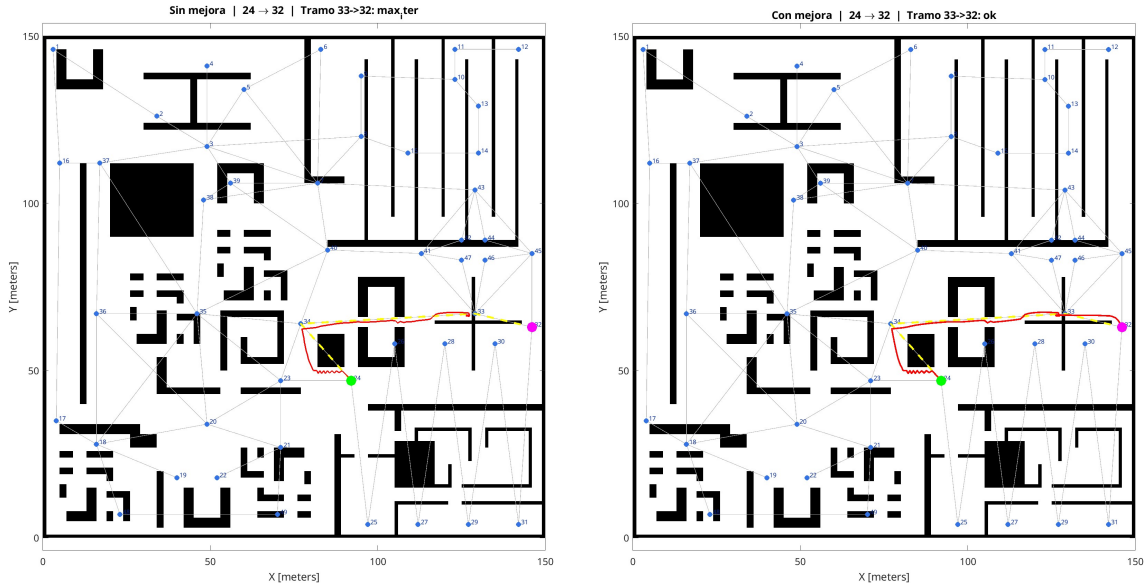


Figura 1: Caso base nodo 1 → nodo 12 con planificación global Dijkstra y navegación local por campos potenciales sobre `mapa2.pgm`. Aristas del grafo en gris, ruta global en amarillo discontinuo, trayectoria del robot en rojo, origen verde y destino magenta.

3.2. Zona conflictiva entre nodos 24 y 32

Al intentar atravesar la región entre los nodos 24 y 32, la versión básica del controlador queda atrapada de forma reproducible: el pasillo es estrecho y los obstáculos están dispuestos simétricamente respecto al eje del robot, con lo que la atracción al subobjetivo y la repulsión combinada se cancelan. Es la situación de mínimo local clásica que ya se identificaba en Lab-MR 3, agravada por la geometría del pasillo.

Mejora propuesta: añadir el campo tangencial F_{tan} (que rompe la simetría empujando al robot hacia el lado del subobjetivo) y, como red de seguridad, una maniobra de escape activada por estancamiento. Con esta extensión el robot supera la zona y alcanza el destino.



(a) Sin mejora: el robot se atasca en la zona conflictiva.

(b) Con mejora: campo tangencial + escape, llega al destino.

Figura 2: Comparativa en la zona conflictiva 24→32: navegación básica frente a navegación con mejora anti-mínimos locales.

3.3. Reemplazo de Dijkstra por A*

Sustituyendo Dijkstra por $\text{aestrella}(C_{\text{eucl}}, H, s, t)$ con la matriz de costes euclídea construida a partir de la adyacencia original y la heurística $H(i, j) = \|p_i - p_j\|_2$, A* devuelve rutas equivalentes a Dijkstra (en coste sobre C^{eucl}) explorando menos nodos, gracias a la información geométrica de la heurística. La consistencia está garantizada por construcción: tanto los pesos como la heurística derivan de la misma métrica.

4. Comentarios finales

La integración jerárquica de planificación global y control local responde a lo que pide el enunciado: la ruta de Dijkstra (o A^*) actúa como secuencia de subobjetivos, y los campos potenciales ejecutan cada tramo evitando obstáculos. Las pruebas en la zona 24–32 confirman que la versión básica falla por mínimo local y que la mejora propuesta (campo tangencial + maniobra de escape por estancamiento) lo resuelve. El reemplazo por A^* funciona correctamente al construir una matriz de costes euclídea coherente con la heurística, condición necesaria para que A^* mantenga la garantía de optimalidad.